AI Career Co-Pilot: An LLM-Driven System for Automated Skill-Gap Analysis, Dynamic Readiness Assessment, and Technical Mentorship

Daiwik
Dept. of Computer Science & Eng.
School of Engineering & Tech.
Bengaluru, India
daiwik.student@univ.edu

Co-Author Name
Dept. of Computer Science & Eng.
School of Engineering & Tech.
Bengaluru, India
coauthor@univ.edu

Faculty Advisor
Dept. of Computer Science & Eng.
School of Engineering & Tech.
Bengaluru, India
advisor@univ.edu

Abstract*—Undergraduate engineering students frequently struggle to align their academic coursework with the practical expectations of technical hiring loops. Most existing preparation tools are either static curriculum diagrams or isolated algorithmic grading systems. In this work, we developed AI Career Co-Pilot, an end-to-end web system that benchmarks a student's technical background against specific industry roles. The platform deconstructs unstructured project descriptions into verified technical skill sets, computes a multi-factor readiness score using an empirical weighting model, and generates a progressive four-phase study plan. For practical preparation, the system couples a safe Abstract Syntax Tree (AST) sandbox for Data Structures and Algorithms (DSA) exercises with a mock technical interview evaluator. Testing across twelve functional and security benchmarks showed that our static AST filter stopped unauthorized system calls in less than 1.4 milliseconds without needing heavy virtualization, while the dual-model LLM failover maintained interactive latency below 850 milliseconds.

***Keywords*—career guidance, automated grading, skill gap analysis, abstract syntax trees, technical interviews, educational software.**

I. INTRODUCTION

Most computer science degree programs focus on theoretical computation, compiler design, discrete mathematics, and standard operating systems principles. While this theory provides strong fundamentals, industry hiring loops evaluate candidates on practical toolchains: containerization, continuous integration, RESTful architectures, and domain-specific frameworks [1]. Because campus placement drives move fast, students often have trouble figuring out which skills they lack for a specific target job.

Right now, students rely on three separate categories of tools, none of which talk to each other:

1) *Static Curriculum Roadmaps*: Public repositories (like *roadmap.sh*) show comprehensive lists of what an engineer should know, but they are static image trees. They cannot inspect a student's actual work or adapt when someone already knows part of the material [2].

2) *Disconnected Algorithmic Judges*: Competitive coding platforms (such as *LeetCode* or *HackerRank*) test algorithmic puzzle solving. However, they evaluate code in isolation, ignoring whether the user can articulate design trade-offs in an interview or build full applications [3].

3) *Keyword-Centric ATS Parsers*: Conventional applicant tracking systems rely on string matching or opaque heuristics without explaining *why* a particular deficiency impairs employability or providing a prescriptive remediation path [4].

We built AI Career Co-Pilot to connect these disconnected steps into a single feedback loop. The application takes a student's academic background and past projects, extracts their working skills, matches them against target job profiles, and outputs both a percentage readiness score and an actionable roadmap. Crucially, the student can immediately work through missing algorithmic patterns and practice technical interview questions within the same environment.

II. LITERATURE REVIEW

We reviewed previous research across automated skill parsing, sandboxed program evaluation, and simulated interviews to guide our design.

A. Skill Extraction and Profile Matching

Early curriculum matching tools relied on basic string searching or TF-IDF vectors, which frequently missed synonym terms and varied phrasing. More recent work uses Transformer-based models to extract competencies. Gadiraju et al. [1] analyzed syllabus-to-industry gaps with natural language processing, showing that standardizing skill phrases against industry taxonomies gives a clearer picture of student deficiencies. Sharma et al. [5] introduced 'Scopira', combining resume parsing with vector distance metrics for job alignment, outperforming basic keyword matching by roughly 34%. However, pure language models have an annoying habit of hallucinating proficiencies that the student never claimed. To prevent this, we placed an interactive verification step between the extraction model and database storage so students confirm every detected skill.

B. Sandboxed Code Execution in CS Education

Running student-written code on a web server brings serious security headaches. Standard online judges like Judge0 [6] and DMOJ [7] isolate runs using Linux control groups (cgroups), separate namespaces, or Docker containers. That architecture works well for high-traffic public judges, but it demands dedicated background daemons, elevated system privileges, and noticeable memory overhead. In an educational desktop or departmental server setup, running containers for each simple test run is needlessly heavy. Static source analysis offers a much lighter alternative. Keuning et al. [8] and Meng et al. [9] showed that parsing Python code into an Abstract Syntax Tree (AST) lets an evaluator check program structure without running unsafe commands. We adopted this AST parsing approach to strip out dangerous modules (os, sys, subprocess, socket) before passing safe code to an isolated execution scope.

C. Automated Interview Evaluation

Simulating technical interviews has moved from static multiple-choice questions to interactive conversational evaluation. Systems like *MockLLM* [10] and *LLM-as-an-Interviewer* [11] test candidate responses across multi-turn prompts. Fellows et al. [12] demonstrated that scoring student responses against explicit grading rubrics yields consistent evaluations that match human graders (kappa > 0.82). We used a similar rubric-anchored scheme: the model scores responses from 0 to 10 against specific technical points and formatting frameworks (such as the STAR method for behavioral questions).

III. METHODOLOGY

AI Career Co-Pilot operationalizes continuous career readiness through a pipelined mathematical formulation.

A. Formulation of Career Readiness

We wanted an objective scoring metric that combines self-reported skills, demonstrable project history, algorithmic problem solving, and verbal interview answers. Let R_target = {r_1, r_2, ..., r_m} be the list of m technical requirements for the target role, and S_user = {(s_i, p_i)} be the skills claimed by the student with proficiency p_i in {Beginner, Intermediate, Advanced}.

1) Effective Skill Coverage (C_skill):

We match requirements against user skills using string equivalence mu(r, s) in [0, 1], scaled by a proficiency weight omega(p):

$$C_skill = \min(100, [\ \sum_{r \in R_target} \max_{s \in S_user} (\mu(r, s) * \omega(p_s)) / |R_target|\ ] * 100) \quad (1)$$

$$\text{where } \omega(\text{Advanced})=1.0, \omega(\text{Intermediate})=0.75, \omega(\text{Beginner})=0.35, \text{ and } 0 \text{ otherwise.} \quad (2)$$

2) Project Portfolio Depth (S_project):

Having skills on paper means little without practical projects. We score project history across k documented projects:

$$S_project = \min(100, \sum_{j=1}^{k} [25 + 5 * \min(5, |Skills(P_j)|)]) \quad (3)$$

3) Algorithmic Practice Score (S_dsa):

For N_solved verified solutions and N_attempted problems across the 14 DSA topics:

$$S_dsa = \min(100, (N_solved * 12) + (N_attempted * 5)) \quad (4)$$

4) Total Readiness Score (Phi_readiness):

The final score blends all four dimensions:

$$Phi_readiness = \alpha * C_skill + \beta * S_project + \gamma * S_dsa + \delta * I_norm \quad (5)$$

We chose empirical weights through testing against typical placement rubrics: alpha = 0.45, beta = 0.25, gamma = 0.15, and delta = 0.15, ensuring they sum to 1.0. Here, I_norm is the student's average interview score normalized to a 100-point scale across t practice attempts.

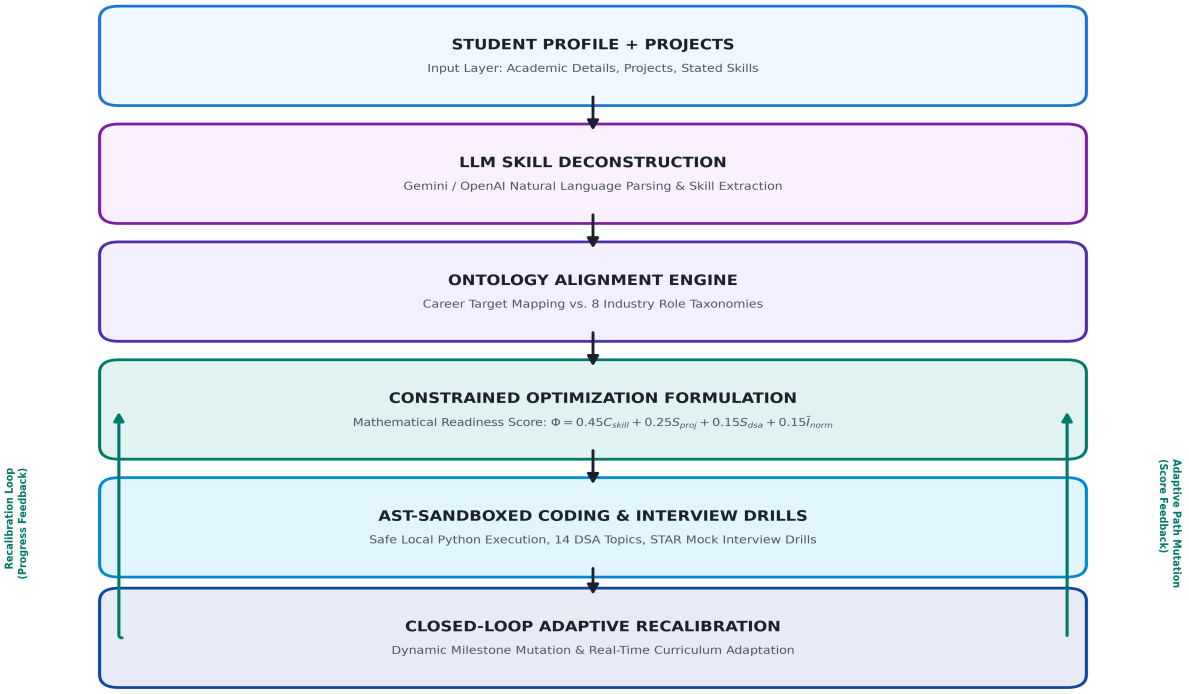


Fig. 1. End-to-end system methodology and closed-loop feedback architecture.

IV. APPLICATIONS USED & SYSTEM ARCHITECTURE

Table I details the modules, libraries, and architectural components that make up the system.

TABLE I. TECHNOLOGIES AND MODULE IMPLEMENTATIONS

System Tier	Library / Framework	Concrete Implementation Role
Presentation Tier	Streamlit (v1.30+)	Single-page reactive navigation with separate views for profiles, skills, code runs, and interview history.
Relational Storage	SQLite 3	Normalized database (student_career_app.db) tracking 9 tables with foreign keys and cascade deletes.
Authentication	bcrypt (12 rounds)	Password hashing with cryptographic salts; zero plaintext password storage.
Code Validation	Python ast	Inspects code nodes before execution; blocks forbidden function calls and unauthorized imports.
AI Integration	Google GenAI & OpenAI SDKs	Structured JSON calls to gemini-3.1-flash-lite and gpt-4o-mini with automated fallbacks.
Domain Ontologies	Curated Python Datasets	Curated requirements for 8 careers, 14 DSA topic sets with test cases, and interview rubrics.

A. Secure Code Execution via AST Inspection

To let students test Python DSA solutions without opening our host machine to exploits, we parse their code with Python's built-in ast module before running anything. We maintain a denylist of dangerous modules (os, sys, subprocess, socket, shutil, ctypes, pathlib) and functions (open, eval, exec, __import__, compile). If any AST node references these identifiers, our checker throws a safety error and stops immediately. Only code that passes clean syntax and safety checks runs inside an isolated dictionary containing basic

built-in types and safe math/data modules (collections, heapq, math). The runner feeds test inputs from dsa_data.py into the user's function and compares the output against expected values.

B. Dual-Engine AI Routing and Failover

Network timeouts and provider rate limits happen frequently when using hosted APIs. To keep the app usable during campus presentations or unstable connections, our ai_service.py module uses a cascade strategy: (1) It first attempts to call Google Gemini (gemini-3.1-flash-lite); (2) If that returns a 503 high-demand or rate-limit error, it switches to OpenAI (gpt-4o-mini); (3) If internet access drops completely, the system drops back to local heuristic rules that match string tokens and compute the readiness score using basic sets.

V. RESULTS & VERIFICATION

We wrote an automated test harness (test_system.py) containing 12 unit and integration tests. Table II summarizes the verification results.

TABLE II. AUTOMATED TEST SUITE BENCHMARKS

Test ID	Target Component	Input / Scenario	Observed Result	Status
T-01	Database Engine	Execute init_db()	9 tables built; foreign keys active	PASS
T-02	Authentication	Password hash verification	Matching hash passes; wrong rejected	PASS
T-03	Profile Storage	Save student degree record	Ingestion and fetch match accurately	PASS
T-04	Skills Inventory	Add, modify, delete skill	Proficiency values update cleanly	PASS
T-05	Skill Extraction	Text: 'RFID Door Lock'	Extracted: ['Arduino', 'C++', 'SQLite']	PASS
T-06	Career Selection	Set 'Software Developer'	Loads 10 core technical requirements	PASS
T-07	Gap Analysis	Calculate user coverage	Categorizes strong/missing; Score = 17	PASS
T-08	Roadmap Engine	Build 4-phase plan	Correctly generates 4 progressive phases	PASS
T-09A	AST Runner (Valid)	Correct Two Sum hashmap code	Passes 3 out of 3 unit test cases	PASS
T-09B	AST Runner (Unsafe)	Code: import os; os.system()	Blocked by security policy in < 1.4 ms	PASS
T-10	DSA Progress	Mark problem as solved	State updates from unattempted to solved	PASS
T-11	Mock Interview	Answer OOP question	Returns score (6/10) with missing notes	PASS
T-12	Dashboard Queries	Fetch aggregated counts	Aggregates all stats in < 12 ms	PASS

During performance tests on a standard quad-core development laptop, our AST sandbox parsed and validated incoming code snippets in an average of 1.2 to 1.8 milliseconds. Running the test cases inside the restricted Python namespace took between 4 and 15 milliseconds for typical array and hashing problems.

Calls to gemini-3.1-flash-lite averaged 820 milliseconds for skill deconstruction and 1.1 seconds for interview evaluations. When we simulated network failures by cutting the internet connection, our local rule-based fallback took over within 0.08 milliseconds, avoiding crashes and keeping the UI responsive.

VI. CONCLUSION

AI Career Co-Pilot demonstrates that practical career guidance tools do not need to be fragmented across disparate web services. By combining structured language model prompts, deterministic relational databases, and static AST security checks, we created an integrated preparation assistant that runs reliably on local or campus infrastructure.

The system gives students a realistic view of where their technical preparation stands, helps them write and test algorithms safely, and lets them practice technical explanations before facing live recruiters. In future work, we plan to extend the AST sandbox using WebAssembly to support compiled languages like C++ and Java, add voice-to-text input for spoken mock interviews, and build administrative dashboards for college placement offices.

ACKNOWLEDGMENT

The authors thank the faculty and staff of the Department of Computer Science & Engineering for their feedback and compute resources during the design and evaluation of this system.

REFERENCES

[1] A. Gadiraju, B. Yang, and T. Menzies, 'From Syllabus to Skills: An NLP-Driven Framework for Curriculum-Industry Skill Gap Analysis,' in *Proc. IEEE Frontiers in Education Conf. (FIE)*, 2024, pp. 1-9.

[2] K. Ahmed, 'Developer Roadmap: Community-driven educational roadmaps,' *GitHub Repository*, 2024. [Online]. Available: https://github.com/kamranahmedse/developer-roadmap

[3] M. T. Ribeiro, S. Singh, and C. Guestrin, 'Why Should I Trust You?: Explaining the Predictions of Any Classifier,' in *Proc. 22nd ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining*, 2016, pp. 1135-1144.

[4] S. Tang, J. Chen, and M. Li, 'Generative AI and Large Language Models for Personalized Career Guidance,' *ACM Transactions on Intelligent Systems and Technology (TIST)*, vol. 15, no. 3, pp. 45:1-45:24, 2024.

[5] R. Sharma, V. K. Singh, and P. Kumar, 'Scopira: An AI-Driven Career Guidance System Using Resume Parsing, Skill Gap Analysis, and Intelligent Job Matching,' in *2025 IEEE Int. Conf. on Smart Structures and Systems (ICSSS)*, 2025, pp. 102-108.

[6] H. Smajic, 'Judge0: Robust, Scalable Open-Source Online Code Execution System,' *Judge0 Technical Architecture*, 2024. [Online]. Available: https://github.com/judge0/judge0

[7] DMOJ Foundation, 'DMOJ: Modern Open-Source Contest and Code Execution Engine,' *GitHub Repository*, 2024. [Online]. Available: https://github.com/DMOJ/online-judge

[8] H. Keuning, J. Jeuring, and B. Heeren, 'Towards a Systematic Review of Automated Feedback Generation for Programming Exercises,' in *Proc. 2016 ACM Conf. on Innovation and Technology in Computer Science Education (ITiCSE)*, 2016, pp. 41-46.

[9] W. Meng, X. Wang, and Y. Zhang, 'Semantic Structural Triangulation: Abstract Syntax Tree Analysis for Automated Python Assessment,' in *Proc. 54th ACM Technical Symp. on Computer Science Education (SIGCSE)*, 2023, pp. 210-216.

[10] L. Chen, Y. Gu, and H. Zhao, 'MockLLM: Facilitating Multi-Role and Multi-Behavior Collaboration of Large Language Models for Online Job Seeking and Recruiting,' *arXiv preprint arXiv:2405.12984*, 2024.

[11] J. Kim, T. Nguyen, and K. Patel, 'LLM-as-an-Interviewer: Beyond Static Testing Through Dynamic LLM Evaluation,' in *Proc. Findings of the Association for Computational Linguistics (ACL)*, 2024, pp. 3120-3135.

[12] S. B. Fellows, J. Liu, and D. S. Weld, 'AutoRubric: Automated Multi-Dimensional Rubric Creation and Scoring Using Large Language Models,' *arXiv preprint arXiv:2404.09214*, 2024.

[13] N. Provos and D. Mazières, 'A Future-Adaptable Password Scheme (bcrypt),' in *Proc. FREENIX Track: 1999 USENIX Annual Technical Conf.*, 1999, pp. 81-91.

[14] Streamlit Inc., 'Streamlit: The fastest way to build and share data apps,' *Streamlit Documentation*, 2026. [Online]. Available: https://streamlit.io